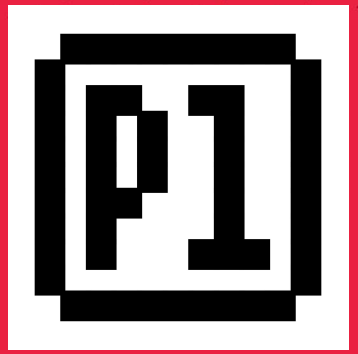
Punteros



Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

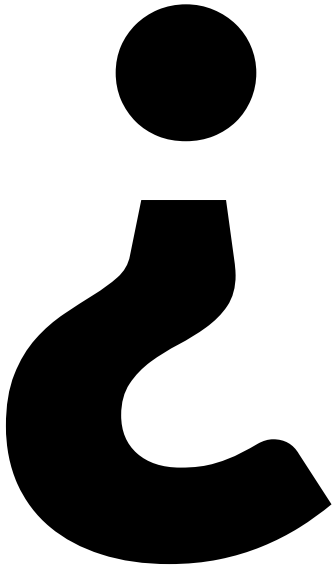
r33

11 / 2026


NYPD

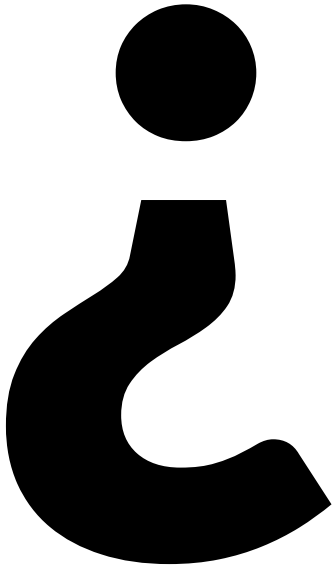
punteros

UNRN Universidad Nacional de Río Negro



qué son





Ya lo vimos indirectamente



Con los arreglos

y su degradación

UNRN | Universidad Nacional de Río Negro


Arrays
Pointers

O, porque un arreglo cambia fuera, pero dentro de una función

Para una función del tipo

```
void funcion(size_t tamanio, int arreglo[])
```

Tenemos esta expresión *casi* equivalente

```
void funcion(size_t tamanio, int *arreglo)
```

`tipo *`

Puntero a



Declaración de una variable de tipo puntero a

```
tipo *identificador;
```



```
int *ptr;
```

¿Y como le decimos a donde apuntar?

Una variable ‘puntero a’ `int` llamada `ptr`

Ojo que, un puntero como r-value **puede** recibir números

```
ptr = 10;
```



¡Pero no debe! Porque el contenido de la variable **no es** un

`ptr` contiene una dirección de memoria

Asignación con la dirección de otra variable

```
int valor = 10;
int *ptr = &valor;
```

¡Los `tipos` de lo apuntado y el 'apuntado' deben coincidir!

&variable
Operador “dirección de”

UNRN Universidad Nacional de Río Negro

¡Ah, como en ¡scanf!

La inicialización en ‘cero’

```
int *ptr = NULL;
```

Apuntando a ‘ningún’ lado. Está definido en `stddef.h`

¿Pero que podemos hacer?

```
int var = 10;
```

1

```
int* ptr = &var;
```

2

```
*ptr = 30;
```

3

```
int var = 10;

int* ptr = &var;

*ptr = 30;
```

una variable

la dirección de

el contenido de


¡Una modificación indirecta!
como los arreglos


¿Preguntas?

*puntero
Operador “contenido de”

UNRN Universidad Nacional de Río Negro

Un ejemplo múltiple

```
int contador = 10;

int *ptr_contador = &ptr_contador;
*ptr_contador = *ptr_contador + 1;
```

Inspeccionando punteros

```
int numero = 10;
int *puntero = &numero;

printf("direccion de numero  %p\n", &numero);
printf("contenido de puntero %p\n", puntero);
printf("direccion de puntero %p\n", &puntero);
printf("apuntado por puntero %d\n", *puntero);
```

De paso vemos `%p` de `printf`

¡A la terminal!



El tamaño de los punteros

Es siempre el mismo

```
sizeof(int*) == sizeof(char*)
```

(Igualito a los arreglos en funciones :-O)

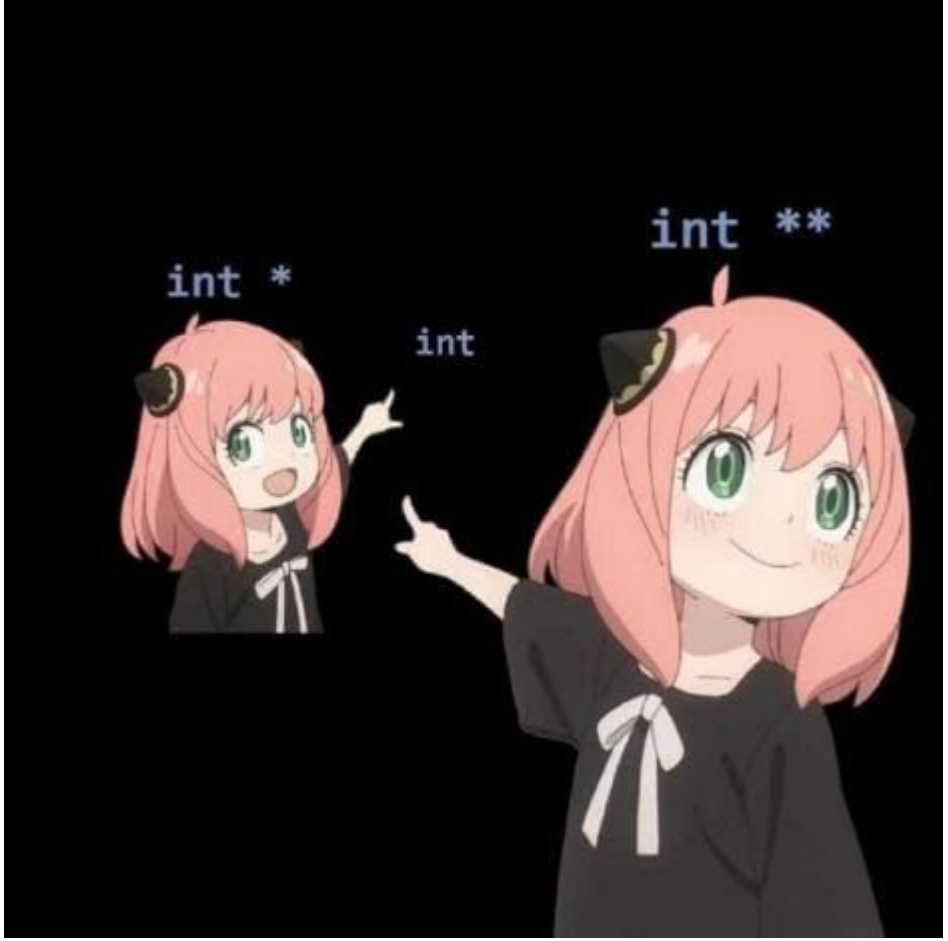
Ojo con el *

```
int contador = 10;

int *puntero = &contador;

*puntero = *puntero * 2;
(*puntero)++;
```

En definitiva



Vía BrunoC [2024]


¿Preguntas?

Efectos en el pasaje de argumentos

UNRN Universidad Nacional de Río Negro

Siempre y solo por valor

Pero...

¿si pasamos un puntero?

Y por referencia si pasamos un puntero

Pero la referencia, en sí, va por valor igual

De-referenciación en funciones

```
void intercambio(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

¡A la terminal!

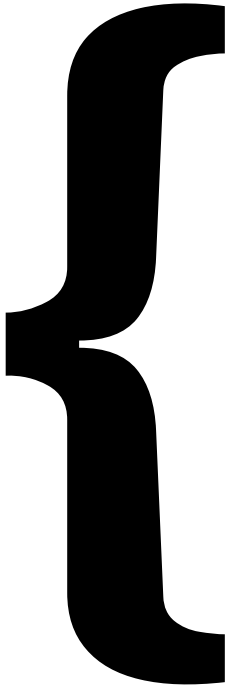


Documentando pasaje por referencia

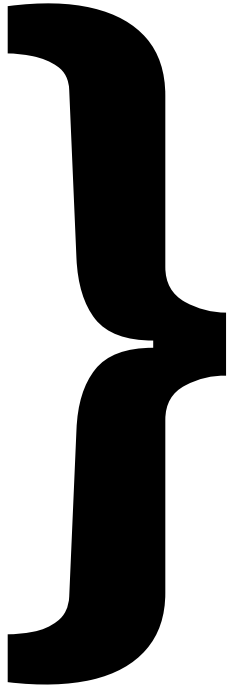
```
/**
 * Este procedimiento toma dos punteros a enteros y intercambia los
 *   valores a los que apuntan.
 *
 * @param izquierdo es un puntero al primer entero.
 * @param derecho es un puntero al segundo entero.
 *
 * @pre Los punteros a 'izquierdo' y 'derecho' deben apuntar a
 *               direcciones de memoria válidas.
 *
 * @post Después de llamar a esta función, los valores apuntados
 *           por 'izquierdo' y 'derecho' habrán sido intercambiados.
 *
 */
void intercambio(int *izquierdo, int *derecho);
```


PUNTEROS
HD
HISTORY.COM
imgflip.com

Otro uso...



Retornos en parámetros



Evolucionando la división lenta

¿Cómo indicamos una división por cero?

```
int division_lenta(int dividendo, int divisor);
```

Evolucionando la división lenta

EDIVZERO

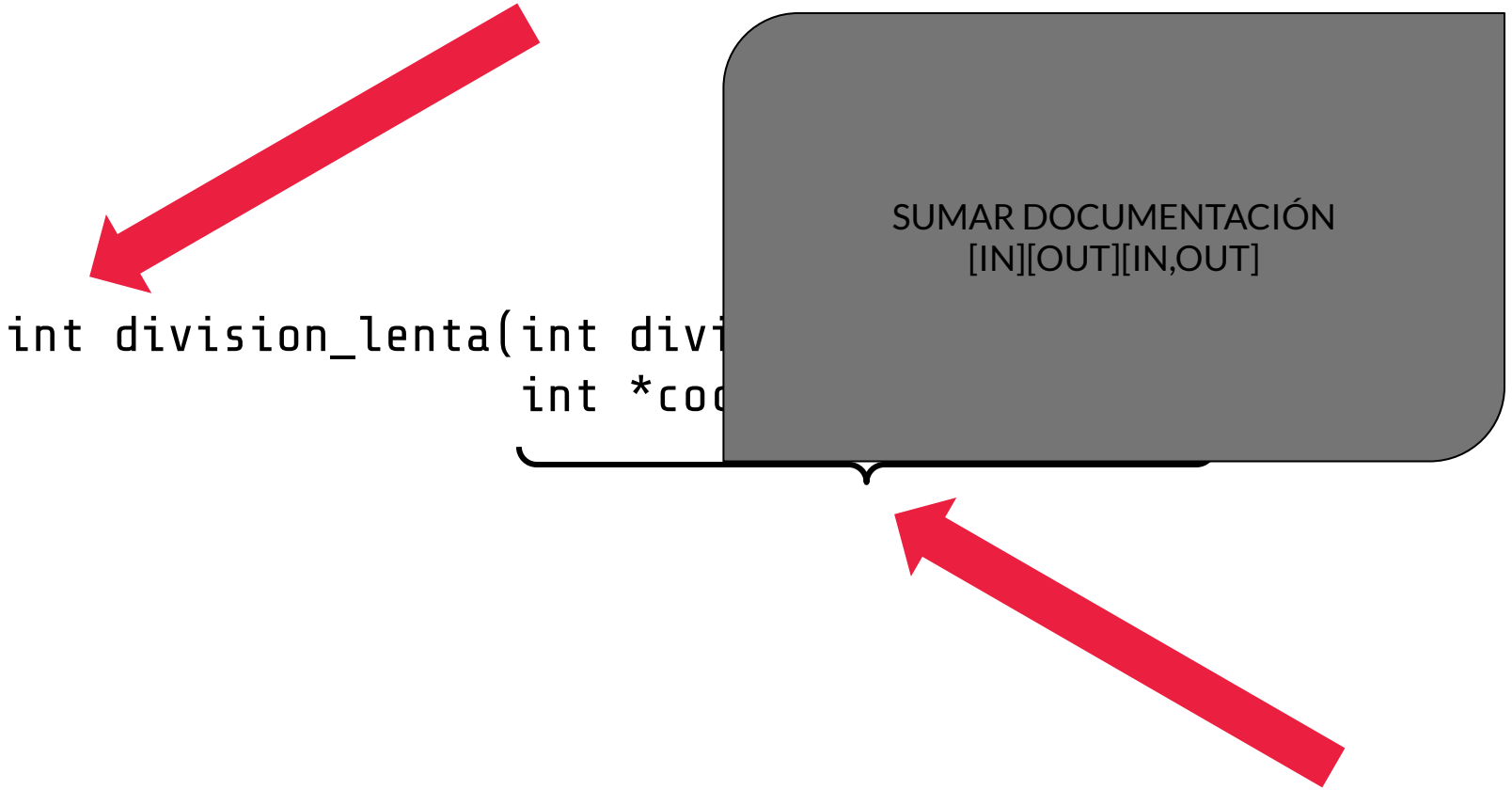
¿Cómo indicamos una división por cero?

```
int division_lenta(int dividendo, int divisor, int *cociente);
```

¡Pero podemos aprovechar y hacer todo junto!

```
int division_lenta(int dividendo, int divisor,
                   int *cociente, int *resto);
```

Devolviendo tres resultados desde una función



Y se llamaría

```
int dividendo = 10;
int divisor = 7;
int cociente;
int resto;
int estado;
estado = division_lenta(dividendo, divisor, &cociente, &resto);
```


¿Preguntas?

También puede ser al revés

```
int division_lenta(int dividendo, int divisor,
                   int *status);
```



Los punteros nos ‘liberan’ el retorno

que podemos usar para gestión de errores

Pero cuidado con ‘mezclar’ la dirección de la información

```
void division_lenta(int dividendo, int divisor,
                    int *cociente, int *status);
```



{ Usando efectos secundarios }

Arreglos II

UNRN Universidad Nacional de Río Negro

Aunque *técnicamente* vimos punteros en arreglos

Y siendo T un tipo de C...

Son parecidos pero *no iguales*

$$T[] \neq T*$$

No podemos usar el arreglo a como R-Value

```
char[] a ≠ const char* a
```

Libertad de asignación

Un puntero puede reasignarse para apuntar a diferentes ubicaciones en la memoria.

Pero un arreglo está asociado con una ubicación específica en la memoria y no puede reasignarse.

const + punteros

en funciones

const T*

Un puntero cuyo contenido es constante

const T*

```
int val1 = 10;
int val2 = 20;

const int* ptr = &val1; // Puntero a un int constante

// Ejemplo 1: No se puede modificar el valor apuntado
// *ptr = 15; // Esto dar un error de compilación

// Ejemplo 2: El puntero puede apuntar a otro lado
ptr = &val2; // Esto vá
```

T* const

Un puntero cuya direccion es constante

T* const

```
int val = 10;
int val2 = 20;
int* const ptr = &val; // Puntero constante a un int

// Ejemplo 1: Se puede modificar el valor apuntado
*ptr = 15; // Esto vá

// Ejemplo 2: No se puede cambiar a quién apunta
// ptr = &val2; // Esto dá un error de compilación
```

const T* const

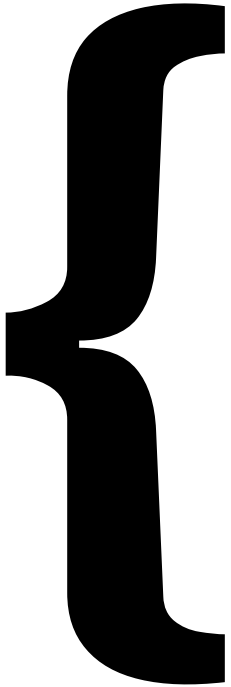
Un puntero cuya dirección y contenido son constantes.

const T* const

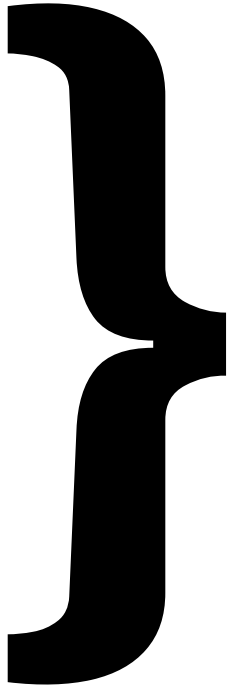
```
int val = 10;
int val2 = 20;
const int* const ptr = &val; // Puntero constante a un int
constante

// Ejemplo 1: No se puede modificar el valor apuntado
// *ptr = 15; // Esto da un error de compilación ❌

// Ejemplo 2: No se puede cambiar la dirección
// ptr = &val2; // Esto da un error de compilación ❌
```



Entonces



T[] = T* const


¿Preguntas?

constante

adj. Dicho de cosa: Que se mantiene invariable.

Aplica a variables y argumentos

```
const int VALOR = 5;
VALOR = 10; // Error: VALOR es constante
```

En argumentos es *particularmente* útil

```
int largo_seguro(const size_t capacidad, char *cadena);
```

Para aquellos parámetros que no deben cambiar accidentalmente y no darle *más de un rol* a los mismos.


¿Preguntas?

Aritmética de punteros

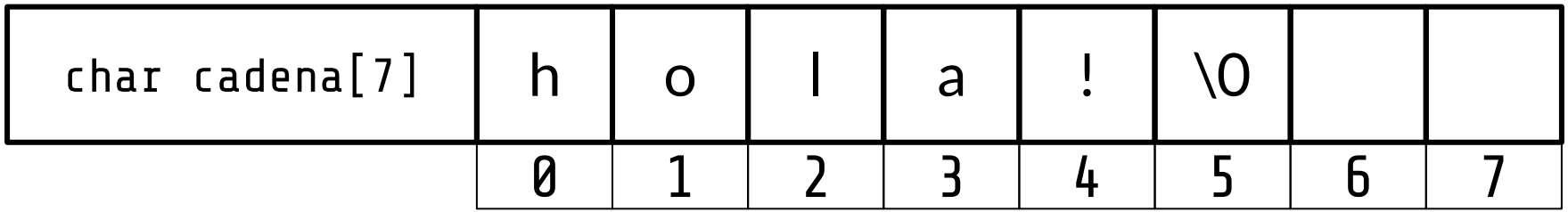
UNRN Universidad Nacional de Río Negro

¡Con cuidado!

Que sí nos pasamos…
con suerte nos enteramos

Hasta acá, lo normal

char cadena[7]	h	o	l	a	!	\0		
	0	1	2	3	4	5	6	7



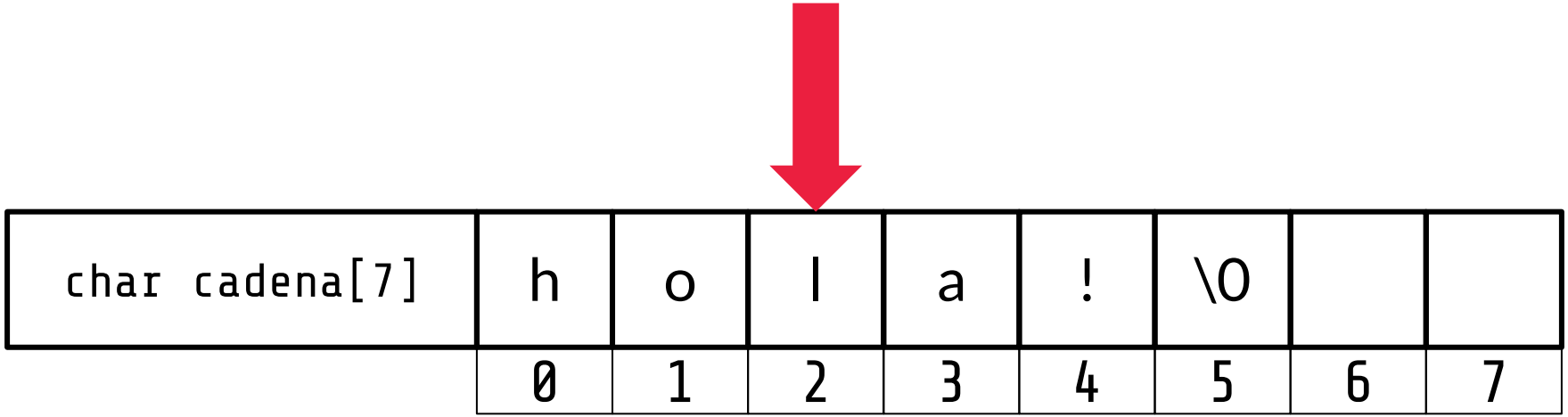

```
char *puntero = arreglo;
```

Pero...

Arreglos a la fuerza

```
char cadena[7] = "Hola!";
char* puntero = cadena; // lo 'degradamos' a mano

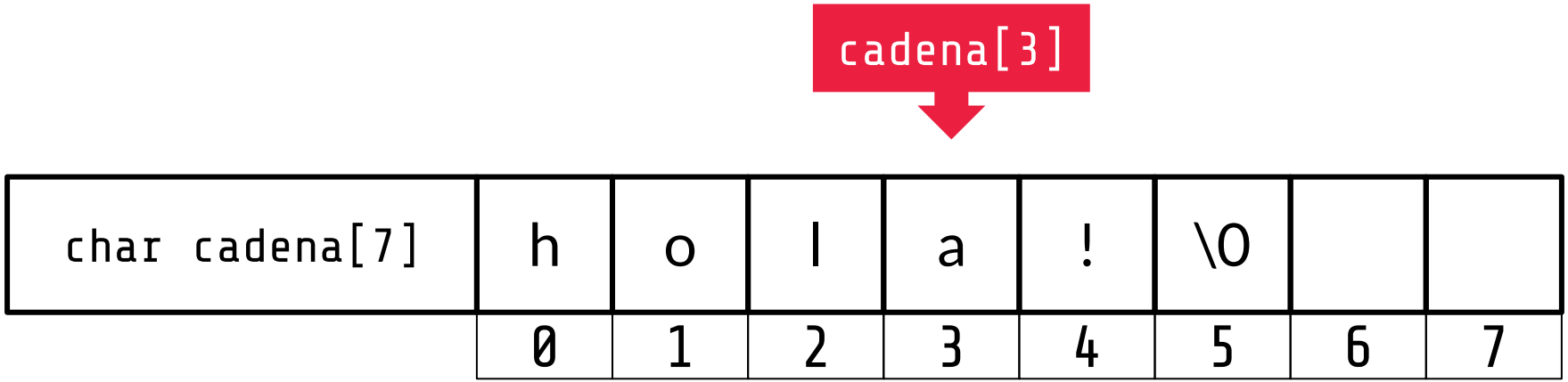
printf("%s\n", puntero+2);
```




```
char *puntero = arreglo;
puntero+2
```

¿`cadena+3` es igual a `cadena[3]`?

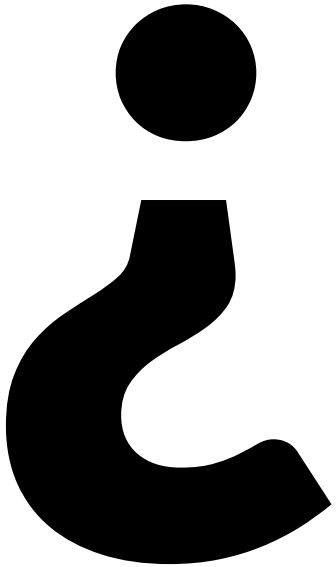
¿*(cadena+3) es igual a cadena[3]?




```
char *puntero = arreglo;
```

¡A la terminal!





```
cadena[3]
==
cadena + 3
```



```
cadena[3] ⇨ char
```

≠

```
cadena + 3 ⇨ char*
```

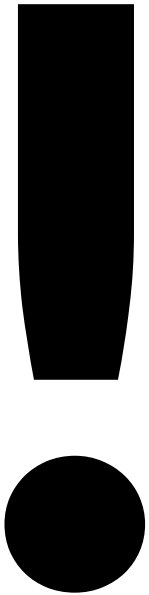
```
*(cadena + 3)
```



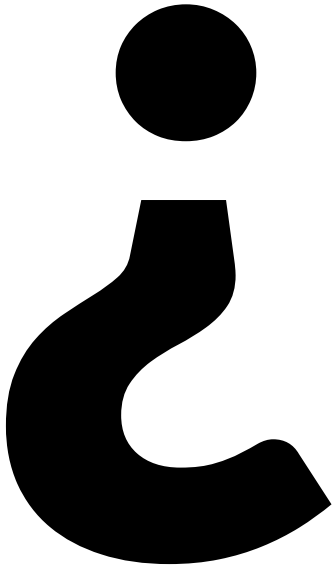
cadena[3]

==

*(cadena + 3)




¿Preguntas?



Y esto para qué



Dada esta función para cadenas seguras

```
int copiar(int capacidad_origen, char origen[],
           int capacidad_destino, char destino[],
           int cantidad);
```

Podemos implementar

```
int copia_segmento(int capacidad_origen, char origen[],
                   int capacidad_destino, char destino[],
                   int desde, int cantidad)
{
    return copia(capacidad_origen - desde, origen + desde, \
                 capacidad_destino, destino, cantidad);
}
```

Aunque incompleto, lo grueso ya está resuelto

{ Con cuidado podemos cambiar el inicio de un arreglo }


¿Preguntas?

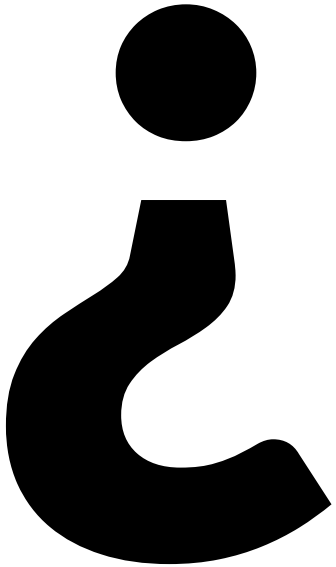
La clase pasada vimos que

```
char cadena[] = "Hola Mundo";

char* ptr_cadena = cadena;

ptr_cadena++; // -> "ola Mundo"
*(ptr_cadena++) // ?
```


¿Preguntas?



Y si sumamos



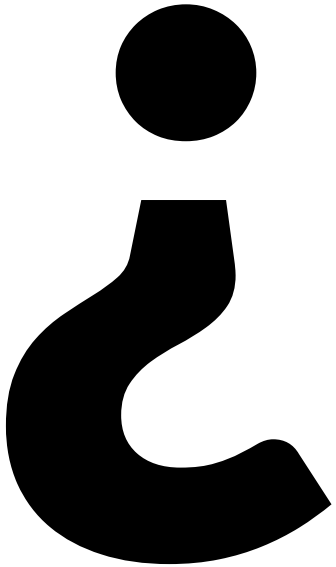
Ya vimos que

```
char arreglo[] = "Hola Mundo";

char* ptr_cadena = cadena;

*ptr_cadena + 2;
*(ptr_cadena + 2);
```


¿Preguntas?



Y si restamos



A cuantos elementos de distancia están ambos

```
int arreglo[] = {10, 20, 30, 40, 50};
int *inicio = arreglo;        // Apunta a 10
int *fin = arreglo + 4;       // Apunta a 50

ptrdiff_t distancia = fin - inicio; // distancia es 4
```

`ptrdiff_t` es parecido a `size_t` pero con signo


¿Preguntas?

Más detalles

1

La aritmética de punteros en punteros que no son relativos al mismo arreglo es comportamiento no definido.

Más detalles

2

Ir a una posición negativa en un arreglo dará comportamiento no definido.

¿Y en este caso?

```
int *puntero;
int numero = *puntero;
```


¿Qué va a pasar?

Punteros salvajes



UNRN Universidad Nacional de Río Negro

Puntero salvaje

Es cuando utilizamos un puntero sin inicializar

¡A la terminal!



¿Y en este caso?

```
int* funcion(){
    int valor = 10;
    return &valor;
}
```

Punteros colgantes

(dangling)



UNRN Universidad Nacional de Río Negro

UNRN Universidad Nacional de Río Negro

Puntero colgante

Es cuando retornamos un puntero a una variable local

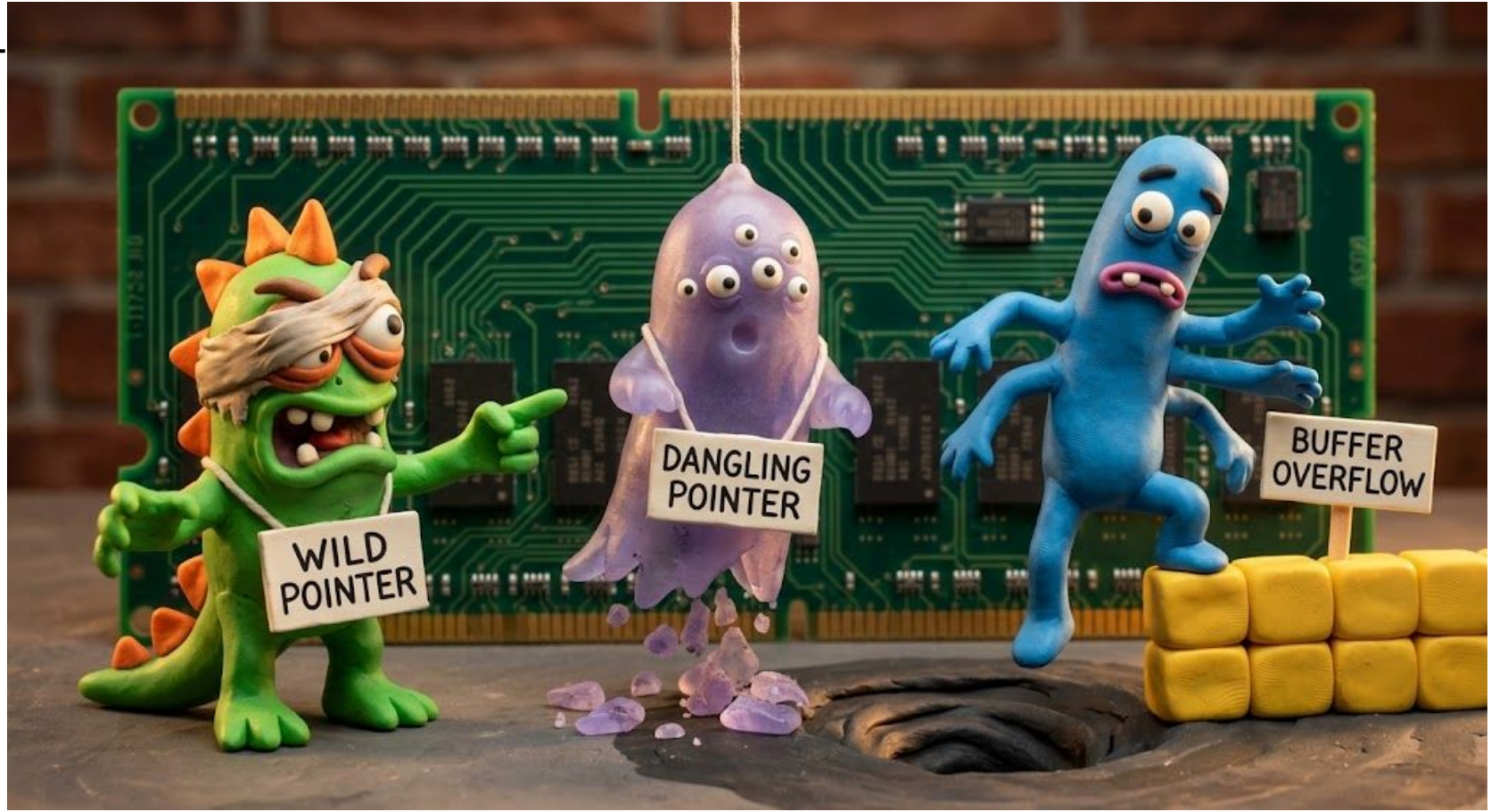
Mala desreferenciación de punteros



UNRN Universidad Nacional de Río Negro

Mala desreferenciación de punteros

Cuando leemos o escribimos fuera de las direcciones válidas



La triada del peligro

¡A la terminal!



Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

unrn.edu.ar

UNRN Universidad Nacional de Río Negro

| unrionegro